Agile Agentic ASPICE (AAA)

From "Vibe Coding" to an Autonomous, Control-Loop-Governed AI Software Factory

Scrum Sprints · 5 ASPICE Stages · Independent Quality Gates · 3-Loop Cascade Control

1. Process Guardrails

5 coupled ASPICE stages with independent **Quality Gates** prevent shortcuts, scope creep, and untested code.

2. Cascade Control

Fast inner feedback loops (compiler, tests, gatekeeper agent2) reject hallucinations and drift **before** human review.

3. Time Leverage

Only **2–3h of focused human time** per sprint instead of constant 15-minute wait loops. Agents work **up to 24h/day**.

Rainer Blind | Architecture & Process Blueprint (`aTrainingTracker`)

November 10, 2026

The Core Idea: Why AAA (*Agile Agentic ASPICE*)?

1. Agile (Scrum Cadence)

- **Iterative Sprints** with strict backlog prioritization (*Rank Discipline*).
- **Ceremony 1 (Start):** Joint ticket screening (Human + Agent).
- **Strategy A:** Shared `sprint/<ID>` branch prevents merge hell.
- **Ceremony 2 (End):** Live on-device review & *Retro Before Merge*.

2. Agentic (Separation of Powers)

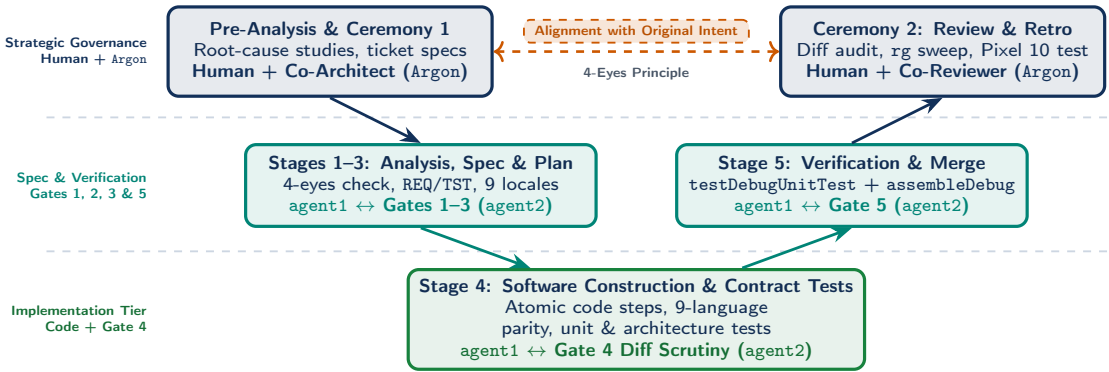
- **Product Owner (Human):** Vision, priorities & final UX sign-off.
- **Co-Architect (`Argon`):** Root-cause studies, Ceremony 1 & 2 audit.
- **Zipper Pair (`pro` ↔ `flash`):** Alternating `agent1` (Doer) & `agent2` (Gate).
- **Epic-Scoped Persistence:** One persistent pair per Epic across the sprint.

3. ASPICE (Speed Bumps)

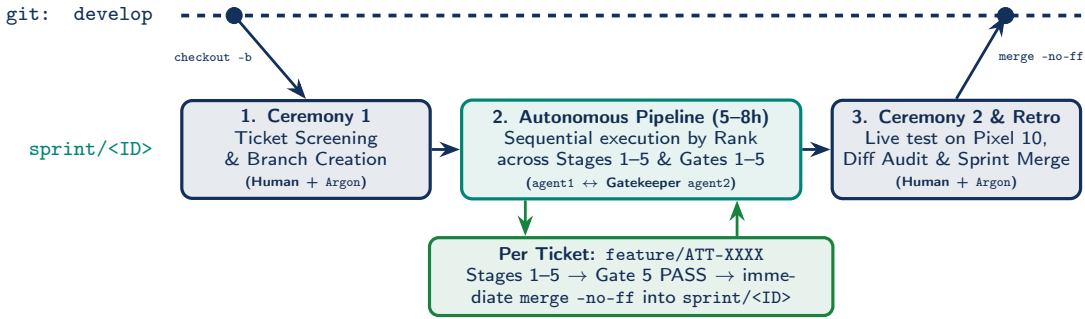
- **Deliberate "Speed Bumps":** No code before analysis, spec & plan.
- **5 Coupled Sub-Tasks** governed by strict Quality Gates (Gates 1–5).
- **Chesterton's Fence:** Mandatory proof before altering existing REQs.
- **Infinite Context:** Living docs serve as permanent project memory.

Key Insight from Automotive Engineering: In the automotive industry, A-SPICE introduces deliberate process guardrails so engineers cannot cut corners on safety-critical software. For AI agents operating at 100× speed—which naturally gravitate toward the path of least resistance—these **speed bumps are the essential safety net**.

Workflow 1: The Hybrid V-Model (Human & Argon ↔ Autonomous Pipeline)



Workflow 2: The Scrum Sprint & Git Branching Flow (Strategy A)



Sprint Capacity & Complexity Split Guardrail (1/2/3/5+ Story Points)

1 SP: Trivial / Isolated

Scope: Single-file fix, string/copy, icon color, simple guard clause.

Agent Pipeline: ∼20–30 min

Human Review: ∼1–2 min

Action: Ready for Sprint

2 SP: Standard Atomic

Scope: Well-bounded change across 2–4 files (ViewModel + Compose + Tests).

Agent Pipeline: ∼35–45 min

Human Review: ∼3–5 min

Action: Ideal Sprint Ticket

3 SP: Complex / Cross-Layer

Scope: DB schema migration, sensor state machine, complex chart math.

Agent Pipeline: ∼50–65 min

Human Review: ∼5–8 min

Action: Sprint Cap (Max 2–3)

$\geq$ 5 SP: Too Large (Mini-Epic)

Scope: Multi-screen workflow, multi-layer service rewrite, complex epic.

Agent Pipeline: $>$ 75 min (high risk)

Human Review: $>$ 10 min

Action: MANDATORY SPLIT!

1. Human Verification Capacity (`customfield_10016`):

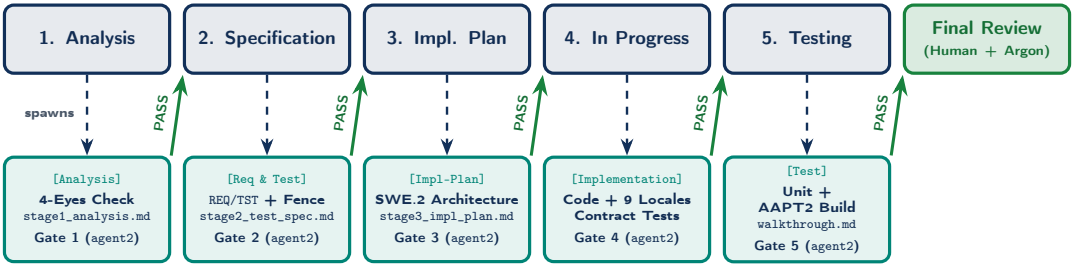
In AAA, coding time is cheap (agents work 24h/day), but Human Final Review & On-Device Verification is the scarce bottleneck. Target sprint load: ∼24–30 SP (∼12–15 atomic tickets = ∼30–45 min focused human review).

2. Automated Complexity Split Guardrail (`REQ-PRO-029`):

Enforced in `jira_util.py move KEY analysis`: Any ticket estimated at $\geq$ 5 SP is strictly blocked from sprint execution. It must be decomposed into 1–3 SP child tickets during Ceremony 1 Screening, preventing context bloat and multi-concern PRs.

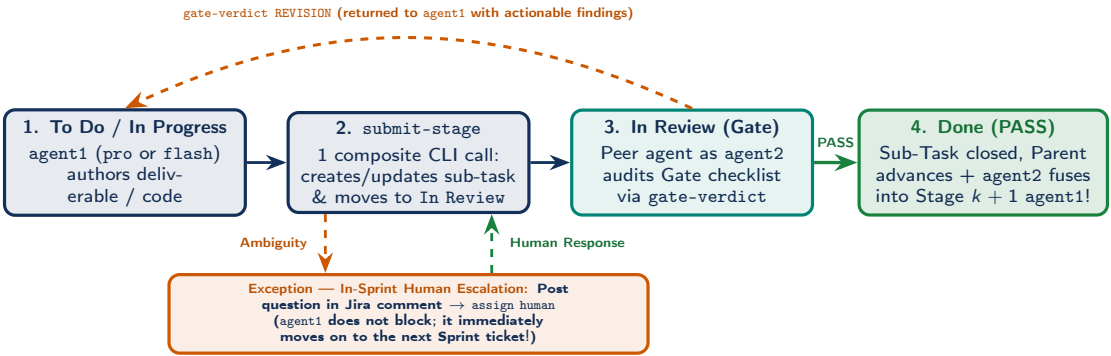
Workflow 3: The 2-Tier Cascade (Parent Ticket ↔ 5 Stage Sub-Tasks)

Tier 1: Parent Ticket Status in Jira (blocked until the corresponding Stage Sub-Task reaches `Done`)



Tier 2: The 5 Coupled Stage Sub-Tasks (`agent1` produces artifact/code → independent `agent2` audits Gates 1–5)

Workflow 4: The Sub-Task State Machine (incl. Revision & Escalation)



Agent Creation & Selection: The "Zipper Principle" & Epic Persistence

1. Model Selection: The "Zipper Principle" (Rules 7 & 60)

Two Complementary Models per Pair: `subagent_pro` (***deep reasoning***) + `subagent_flash` (***fast synthesis & test runs***).

Stage	Doing (`-as agent1`)	Gate (`-as agent2`)
S1: Analysis	`pro` **(RCA & blast radius)**	`flash` **(Gate 1 audit)**
S2: Spec	`flash` ***(fused with G1!)***	`pro` **(Gate 2 + Fence)**
S3: Plan	`pro` ***(fused with G2!)***	`flash` **(Gate 3 audit)**
S4: Code	`flash` ***(fused with G3!)***	`pro` **(Gate 4** `git diff`**)**
S5: Verify	`flash` **(Suite + Auto-Verify)**	`pro` **(Gate 5 + Merge)**

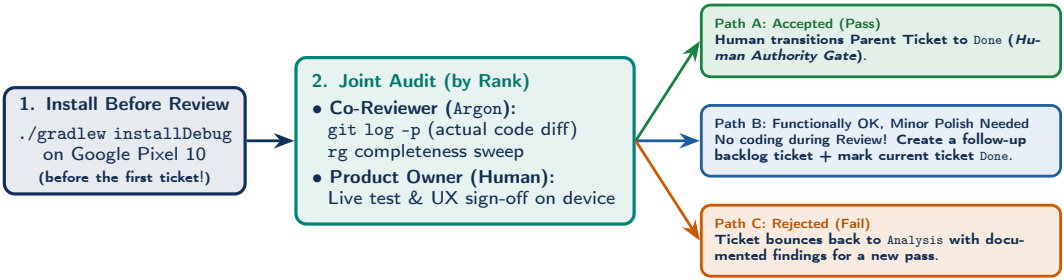
→ **Fused Gate-to-Doing Handoffs: When the reviewer passes Gate k, its context window is already 100% warm with the ticket's files—so it immediately executes Stage $k+1$ Doing in the same turn (cutting 10 turns/ticket down to 5).**

2. Agent Creation: Epic-Scoped Persistent Pairs (`ATT-4520`)

- **Cold-Start Bottleneck:** Spawning 2 fresh agents per ticket wastes ∼8–10 min re-reading governance & code.
- **Epic-Scoped Agent Pairs (`1 Pair per Epic`):** Per sprint Epic (`ATT-111`, `ATT-281`, `ATT-355`), the Coordinator spawns **one persistent** (`pro`, `flash`) **pair** reused via `send_message` across **all tickets in that Epic**.
- **Cumulative Domain Memory:** By ticket 2+, both agents already hold the module's repositories, SQLite schemas, and Compose patterns in context.
- **Clean Lifecycle:** Pairs stay alive across all Epic tickets (incl. bounce-backs) and terminate at Sprint End.

Measured Velocity Gain: Combining **Zipper Fused Turns** (5 turns instead of 10), **Composite CLI Calls** (`submit-stage` / `gate-verdict`), and **Epic-Scoped Persistent Pairs** reduces cycle time from **51.5 min/ticket** down to **35–44 min/ticket** while preserving strict 4-eyes independence.

Workflow 5: Ceremony 2 — Joint Review & Decision Tree



After the Final Ticket — Mandatory Retrospective (*Retro Before Merge*): Before merging `sprint/<ID>` into `develop`, Human and `Argon` jointly analyze: Which issues slipped past the automated checks and quality gates (Gates 1–5)—and how do we sharpen our skills and gate rules for the next sprint?

Detailed Gate Criteria: Stage 1 (Gate 1) & Stage 2 (Gate 2)

Stage 1: [Analysis] (pro) → Gate 1 (flash)

Artifact: ATT-XXXX_analysis.md | Analysis → Specification

Role of agent1 (subagent_pro — 4-Eyes Principle): Validates the ticket pre-analysis as a hypothesis against the actual code on sprint/<ID>; identifies all call sites, existing REQ IDs, and system invariants.

Gate 1 Checklist (agent2 / subagent_flash via gate-verdict):

1. **Root Cause vs. Symptom:** True cause proven in code?
2. **User Scope Grounding:** No unrequested rewrites?
3. **Blast Radius:** All call sites & affected modules listed?
4. **System Invariants:** Stated what *must not* break?
5. **Risk Classification:** LOW / MEDIUM / HIGH justified?

Stage 2: [Req & Test Spec] (flash) → Gate 2 (pro)

Artifact: ATT-XXXX_test_spec.md | Specification → Impl. Plan

Role of agent1 (subagent_flash — Living Docs & 9 Locales): Updates requirements.md (REQ-XXX), tests.md (TST-XXX), defines *Given-When-Then* criteria, and plans string resources across all 9 languages.

Gate 2 Checklist (agent2 / subagent_pro via gate-verdict):

1. **Atomic Syntax:** SHALL/MUST without implementation leaks?
2. **Bidirectional Traceability:** Every REQ-XXX ↔ TST-XXX?
3. **Chesterton's Fence (4-Field Mandate):** Whenever existing REQs are modified or removed, *Original ID*, *Git Commit/Ticket*, *Historical Reason*, and *Invariant Preservation* must be proven—otherwise REVISION!

Detailed Gate Criteria: Stages 3, 4 & 5 (Gates 3, 4 & 5)

Stage 3 (pro) → Gate 3 (flash)

Sub-Task: [Impl-Plan]
Artifact: ATT-XXXX_plan.md

- **SWE.2 Architecture:** Layer & component boundaries respected?
- **DB Schema Safety:** Parity between onCreate and onUpgrade?
- **Test Mapping:** Every atomic step mapped to a test in tests.md?
- **Code Lock:** Zero production code changes allowed during Stage 3!

Stage 4 (flash) → Gate 4 (pro)

Sub-Task: [Implementation]
Focus: Code, Tests & 9 Locales

- **Diff Scrutiny:** Does actual git diff match the Stage 3 plan?
- **No Scope Creep:** No unrelated files or unauthorized refactoring?
- **Contract Tests:** Guards against partial refactoring green?
- **9-Locale Parity:** All 9 language files (EN..ZH) completely updated?

Stage 5 (flash) → Gate 5 (pro)

Sub-Task: [Test]
Artifact: ATT-XXXX_walkthrough.md

- **Unit Tests:** Tier 1 testDebugUnitTest 100% green (0 failures)?
- **Resource Build:** assembleDebug (AAPT2) 100% error-free?
- **Living Docs:** Auto-flipped to Verified on submit-stage 5?
- **Sprint Merge:** 1 squashed commit merged into sprint/<ID>?

Infinite Agent Context & Why We Do Not Allow a "Fast-Track"

1. Artifacts as "Infinite Agent Context"

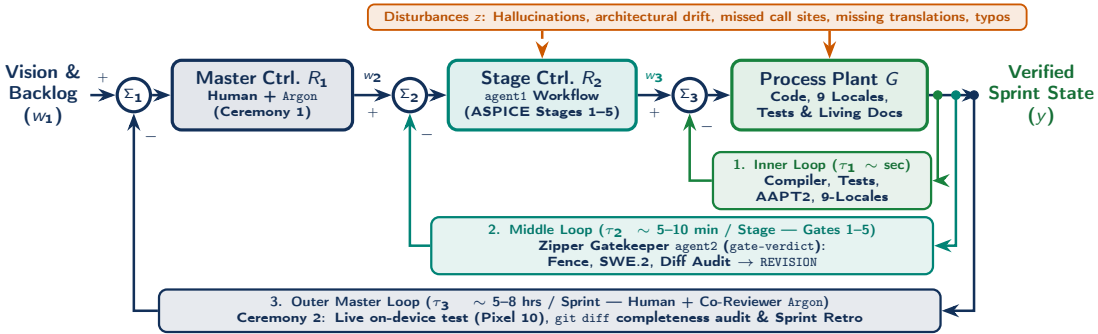
- **The Problem of Ephemeral Chats:** An LLM context window ends with the session—implicit knowledge discussed only in chat is lost forever.
- **The Repository as Long-Term Memory:** Through `docs/workflows/`, `requirements.md`, `tests.md`, and `walkthroughs/`, a permanent knowledge base lives directly in Git.
- **Real-World Leverage:** When analyzing new bugs or features, agents automatically read prior walkthrough and spec documents, immediately grasping the historical *"Why"*.

2. Why There Is No "Fast-Track"

1. **Slippery-Slope Effect:** Once a "Fast-Track" for supposedly trivial tickets exists, convenience ensures it gets used every time—eroding process discipline immediately.
2. **Zero Agent-Initiated Shortcuts:** Agents must never propose skipping stages or quality gates on their own initiative.
3. **Mandatory Pushback:** If the human requests an exception, the agent *must* first object with a reasoned risk assessment (documented deviation).

Result: Maximum autonomous development velocity (24/7) with full automotive-grade traceability and institutional memory.

Control Engineering Principle: Agile Agentic ASPICE as Cascade Control



Why Cascade Control ($\tau_1 \ll \tau_2 \ll \tau_3$)? Disturbance Rejection Before Reaching the Human! Fast inner loops (τ_1: compiler/tests, τ_2: independent gatekeeper agent2) reject disturbances (z) right at the source **before** they propagate to the slow outer master loop (τ_3). That is why the human does not need to intervene every 10 minutes!

Time Allocation: Vibe Coding vs. Agile Agentic ASPICE

A. Classic "Vibe Coding": Human Trapped in a 15-Minute Wait Loop (Active Only a Few Hours/Day)

Human:	1. Prompt	Waits	2. Review	Waits	3. Prompt	Waits	4. Review	**18–20 Hours / Day Standstill** (Zero progress whenever human leaves desk)
Agent:	Idle	5–20 min	Idle	5–20 min	Idle	5–20 min	Idle	

B. Agile Agentic ASPICE: 2–3h Focused Human Time per Sprint — Up to 24h/Day Autonomous Agent Execution

Human (2–3h):	Ceremony 1 (∼1h) Ticket Screening	5–8 Hours Completely Free (Day Job, Sports, Sleep) or asynchronous brainstorming in Chat 2	Ceremony 2 (∼1.5h) Review & Retro	Free / Overnight (Sprint $N + 1$ runs)
Chat 1 (Sprint):	Co-Architect (`Argon`)	5–8h Autonomous Execution (Stages 1–5, Gates 1–5) `agent1` ↔ `agent2` (Code, 9 Locales, Tests, Docs)	Co-Reviewer (`Argon`)	Sprint $N + 1$ (5–8h) Up to 24h/Day Active
Chat 2 (Backlog):		Parallel in Chat 2: Brainstorming, Studies & Backlog (prepares Sprint $N + 1$ without disturbing Sprint N)		

The Paradigm Shift: In **Vibe Coding**, the human is reduced to a clocked "agent babysitter". In **Agile Agentic ASPICE**, the human invests only **2–3 highly focused hours per sprint** (screening at start, review & retro at end)—while agents work autonomously for **5–8 hours per sprint** (up to **24h/day**).

The Living Document Triad: Governance, Coding & Design Rules

1. Process & Governance
`.agents/rules/aspice_governance.md`

- **Rule #1 ("Obey the Rules"):** Reload rules + auto-inject stage `SKILL.md`.
- **Code Lock Until Gate 3:** Zero code edits until `check-gate` passes.
- **Human Sovereignty:** Only human sets `Done`; **no coding** during Review.
- **Device Data Guard:** Never run `uninstall` or `pm clear` without `.attbackup`.

2. Architecture & Coding
`docs/coding_standards.md`

- **Reuse Before Create (DRY):** Canonical formatters; no inline `* 3.6f`.
- **Never Sort Display Strings:** Sort raw `Double` (`"2.40 km"` < `"995 m"`!).
- **No Silent {} or Default Traps:** Wire **all** callers for new callbacks.
- **Strict Mocks & 2-Tier Tests:** Ban `returnDefaultValues`; 2-Tier suite.

3. UI, UX & 9 Locales
`docs/design_guidelines.md`

- **"Look Like the Rest":** Shared shapes (`12.dp`), spacing & theme tokens.
- **9 Locales & DE Stress Test:** Positional `%1$s`; `FlowRow` dialog actions.
- **Cardinality Controls:** 2 → Segmented; 3-4 → FilterChip; ≥ 5 → Slider.
- **Adaptive Elevation & Icons:** Relief-scaled (70–200 dp); true vectors.

Self-Improving Institutional Memory (61 Rules Evolved Across Sprint Retrospectives): Every defect or UX inconsistency caught in Ceremony 2 (*Retro Before Merge*) is distilled into a permanent rule in the Document Triad—ensuring the agent team never repeats the same mistake twice.

Empirical Impact: Evolution of Formal Requirements (REQ-*)

